Step 1: Conduct an in-depth review of the smart contract code to identify any potential vulnerabilities or security flaws.

Step 2: Perform security tests and analysis to ensure the protocol is safe from any type of attack.

Step 3: Deliver a detailed audit report outlining the findings, recommendations, and any necessary code fixes

Auditing Process

- Business Logic's Review
- Functionality Checks
- Access Control & Authorization
- Escrow manipulation
- Token Supply manipulation
- User Balances manipulation
- Data Consistency manipulation
- Kill-Switch Mechanism
- Operation Trails & Event Generation

Step 1 : Specification Gathering / Prepare For a Security Audit

This is the most crucial stage because the detail is key for a successful smart contract Security audit. Here is how you can prepare for it:

Code quality • Remove dead code and comments • Consistent coding style. • Follow the Solidity / Rust (Solana) style guide.

Use comments to document complex parts of the code but also make sure these are. consistent with the code

Test the code • Make sure the contracts can be compiled and fully tested. • Perform high coverage and high-quality unit tests.

This will maximize focus on the difficult parts of the code. Auditing should not be discovered that some functions are uncallable, or do not do what they are expected to do under entirely straightforward inputs. Optimal auditing should focus on unexpected, corner-case, possibly adversarial behavior.

Code freeze • Freeze the code and specify the commit hash. Or, deploy the code on testnet and share the link.

Step 2 - Manual Review

Here we would look for undefined, unexpected behavior and common security vulnerabilities. The goal is to get to as many skilled eyes on contract code as possible. Aims of manual review:

- Focus on issues regarding security, attacks, mathematical errors, logical issues, etc.
- Check the code for any vulnerabilities that can be exploited.

- Verify that every detail in the specification is implemented in the smart contract.
- Verify that the contract does not have any behavior that is not specified in the specifications.
- Verify that the contract does not violate the original intended behavior of specifications.

Step 3 - Functional Testing

- The smart contract will be manually deployed in a sandbox environment like testnet/mainnet forks, hardhat, ganache, etc
- Smart contract functions will be tested on multiple parameters and under multiple conditions to ensure that all paths of functions are functioning as intended.
- In this phase, the intended behavior of the smart contract is verified.
- In this phase, we would also ensure that smart contract functions are not consuming unnecessary gas.
- Gas limits of functions will be verified in this stage.

Step 4 - Testing over Latest Attack Vectors

- Team researches newly discovered attacks(like **market manipulation, LP pricing, front running vectors,** and more), and try to replicate them on the project in order to make sure, the project is safe from those attacks
- In case, if the current implementation is found vulnerable to those newly discovered attacks, we recommend the project team switch to a safer implementation.

Step 5 - Testing with Automated Tools

Testing with automated tools is important to catch those bugs that humans miss. Some of the tools we would use are (Based on the requirement/ Auditor preference we use specific tools) :

- Mythril / Mythx
- Solgraph
- Solidity Coverage
- Slither
- Solidity Visual Developer

Step 6 - Initial Audit Report

In the end, we would provide you with a comprehensive report, which we call the Initial Audit Report (IAR):

- A Comprehensive Audit report.
- Encapsulates details of the Audit & solutions to the vulnerabilities (if we found any) in your contracts.
- We expect you to resolve the identified bugs & make suitable changes to the code.

Step 7 - Final Audit Report

After initial audit fixes, the process is repeated, and the Final Audit Report is delivered. There is a possibility that even after the fixes you've made, some issues are still not resolved, and/or those changes have led to a few more issues.

So, after receiving the Final Audit report, you have to take a call (based on the severity table containing the unresolved issues) on whether to alter the code again or to move forward as it is.

Step 8 - Delivery

After getting a green light from the previous step, we send the report to our designers. With their skills, they make a PDF version of the Audit Report and beautifully showcase everything in it.

Tools :

Slither
Mythril

Mythx

Echidna

Manticore

Surya

Remix

Foundry

Hardhat